EDULIBRARY / ТУПНҰСҚА ОҚУ МАТЕРИАЛЫ

Веб-бағдарламалау негіздері

EduLibrary Demo Content

HTTP, семантикалық HTML, бейімделетін CSS, JavaScript және сервер тексеруі арқылы қолжетімді шағын веб-қосымша жасауға кіріспе.

Қысқаша сабақтар · 2026 · Қазақша

Оқу жолы

Бұл материалды қалай оқу керек

Әр сабақтағы ұғымды оқып, мысалды қолмен жүргізіңіз. Содан кейін тәжірибе тапсырмасын өзіңіз орындап, соңғы беттегі түсіндірмемен салыстырыңыз. Код үзінділері оқу үшін ықшамдалған; олар толық өндірістік жүйені алмастырмайды.

1. Браузер мен сервер сөйлесуі

2. Семантикалық HTML және нысан

3. CSS және бейімделетін орналасу

4. JavaScript және асинхронды сұрау

5. Сервердегі шекаралар

Шағын жоба және жауаптарды тексеру

Авторлық құқық және шығу тегі

Автор: EduLibrary Demo Content. 2026 жылы EduLibrary демонстрациясы үшін жаңадан жазылған түпнұсқа қазақша сабақтар. Коммерциялық кітаптан көшірілген мәтін, тарау не файл қолданылған жоқ. Бұл материал бес дербес шағын оқу құралының бірі.

Лицензия: Creative Commons Attribution 4.0 International (CC BY 4.0). Авторды және осы лицензияны көрсетіп пайдалануға болады. Лицензия: https://creativecommons.org/licenses/by/4.0/ . Бастапқы мәтін мен генератор осы жобаның Data/DigitalDemo/content.json және tools/generate_digital_demo.py файлдарында сақталған.

1. Браузер мен сервер сөйлесуі

Браузер URL бойынша HTTP сұрауын жіберіп, серверден жауап алады. Жауапта мәртебе, тақырыптар және дене болады. 200 сәтті нәтижені, 404 ресурс табылмағанын, 403 рұқсат жоқтығын білдіреді. HTML, стиль, JavaScript және суреттер бөлек ресурстар ретінде жүктелуі мүмкін.

GET деректі оқуға, POST күйді өзгертетін әрекетке қолданылады. URL ішіндегі query параметрлер тарихта және журналда қалуы мүмкін, сондықтан құпия кодтарды оған салмаңыз. HTTPS тасымалды қорғайды, бірақ сервердегі рұқсат тексеруін алмастырмайды. Кэш жеке дерекке абайлап бапталуы тиіс.

Мысалды талдау

```
GET /Books?search=SQL HTTP/1.1
Accept: text/html

HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
```

Өзіңіз орындаңыз

Неге броньды тоқтатуды қарапайым GET сілтемесімен жасау дұрыс емес? Браузер не іздеу роботы сілтемені алдын ала ашса не болуы мүмкін?

2. Семантикалық HTML және нысан

HTML құжаттың мағынасын сипаттайды. header, nav, main және section сияқты элементтер құрылымды анық көрсетеді. Тақырып деңгейлері сыртқы өлшемге емес, мазмұн иерархиясына сай таңдалады. Әр енгізу өрісінің label белгісі болуы керек; placeholder оның орнына жүрмейді.

Батырма әрекет жасайды, сілтеме басқа ресурсқа апарады. Пернетақтамен басқару мен анық focus күйі барлық пайдаланушыға көмектеседі. Қате тек қызыл түспен көрсетілмей, мәтінмен де түсіндірілсін. Нысанда name атрибуты серверге жіберілетін өріс атауын анықтайды. Браузердің required тексеруі пайдалы, бірақ сервер де деректі тексеруі міндетті.

Мысалды талдау

```
<form method="get" action="/Books">
  <label for="query">Кітап іздеу</label>
  <input id="query" name="search" />
  <button type="submit">Іздеу</button>
</form>
```

Өзіңіз орындаңыз

Тек белгішесі бар іздеу батырмасына қолжетімді атау берудің екі жолын атаңыз. Label мен input қалай байланысады?

3. CSS және бейімделетін орналасу

CSS мазмұнды көрсету ережелерін береді. Flexbox бір бағыттағы элементтерге, Grid жолдар мен бағандарға ыңғайлы. Бет енін тұрақты үлкен санға бекіту шағын экранда көлденең айналдыру туғызады. minmax(0,1fr), икемді өлшем және дұрыс оралатын мәтін мұны азайтады.

Дизайн токендері түстер мен аралықтарды бірізді ұстайды. Мәтін мен фонның контрастын, батырманың басуға ыңғайлы көлемін және ұлғайтылған қаріптегі орналасуды тексеріңіз. Media query мазмұнға орын жетпеген нүктеде орналасуды өзгертсін. Тек телефон мен жұмыс үстелін емес, аралық ендерді де тексеріңіз.

Мысалды талдау

```
.books {
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));
  gap: 1.5rem;
}
@media (max-width: 700px) {
  .books { grid-template-columns: repeat(2, 1fr); }
}
```

Өзіңіз орындаңыз

Ұзын, бос орынсыз код карточканы кеңейтіп жіберсе, қандай CSS ережесі көмектеседі? Неге мәтінді жасыру әрқашан жақсы шешім емес?

4. JavaScript және асинхронды сұрау

JavaScript пайдаланушы әрекетіне жауап беріп, құжатты жаңарта алады. addEventListener оқиғаны өңдейді, fetch серверге асинхронды сұрау жібереді. Жүктеу кезінде күйді көрсету, қате туралы түсінікті хабарлау және қайталап әрекет ету мүмкіндігін беру маңызды.

Пайдаланушы іздеу мәтінін тез өзгертсе, ескі жауап жаңасынан кейін келуі мүмкін. AbortController немесе сұрау реттік нөмірі осы жарысты басқарады. Серверден келген мәтінді innerHTML арқылы соқыр енгізбеңіз; textContent қауіпсіз мәтін көрсетуге ыңғайлы. Асинхронды қатені catch арқылы өңдеп, түймелерді finally ішінде қалыпқа келтіріңіз.

Мысалды талдау

```
const response = await fetch('/api/slots');
if (!response.ok) throw new Error('Request failed');
const slots = await response.json();
const label = document.createElement('p');
label.textContent = slots[0].label;
document.querySelector('main').append(label);
```

Өзіңіз орындаңыз

Ескі іздеу жауабы жаңа сұраудың нәтижесін басып кетпеуі үшін қандай екі тәсіл қолдануға болады?

5. Сервердегі шекаралар

Клиент жіберген барлық дерек тексерілуі керек. Жасырын input қауіпсіз сақталған мән емес: пайдаланушы оны өзгерте алады. Оқырманның Id мәнін нысаннан емес, ағымдағы Identity пайдаланушысынан анықтау - маңызды үлгі. Ресурсқа қатынағанда оның иесін де тексеріңіз.

MVC архитектурасында controller сұрауды басқарады, service домен ережесін орындайды, view нәтижені көрсетеді. Бұл бөлу ережені басқа интерфейс те қайта қолдануына көмектеседі. CSRF токені бөтен сайттан келген күй өзгертетін сұрауды шектейді. HTML шығару кезінде стандартты кодтауды сақтаңыз; сенімсіз мазмұнды raw HTML етпеңіз.

Мысалды талдау

```
POST /Reader/Progress
AssetId=12&CurrentPage=5

Server checks:
1. Authenticated user
2. Anti-forgery token
3. Asset access
4. Valid page range
5. User-scoped SQL update
```

Өзіңіз орындаңыз

Басқа оқырманның bookmark Id нөмірін білу оны жоюға рұқсат бере ме? Сервер қандай қосымша шартпен іздеуі керек?

Жоба және өзіндік тексеру

Қолжетімді кітап іздеу бетін жасаңыз. Семантикалық нысан JavaScript өшірулі кезде де жұмыс істесін. JavaScript қосылса, жүктеу күйі және жарысқа төзімді асинхронды іздеу қосылсын. 360, 768 және 1440 пиксельде, пернетақтамен және ұзын атаулармен тексеріңіз.

Тапсырмаларға түсіндірме

1. GET қауіпсіз оқылым ретінде қаралады. Автоматты ашу күйді рұқсатсыз өзгертіп кетуі мүмкін. Өзгеріс POST және CSRF қорғанысымен бөлек расталуы керек.

2. Көрінетін мәтін қосу немесе aria-label беру. Label for мәні input id мәніне сәйкес келеді; label ішіне input салу да байланыс жасайды.

3. overflow-wrap:anywhere кодтың жолын орауға көмектеседі. Жасыру пайдаланушыға қажет сәйкестендіргішті оқуға не көшіруге кедергі келтіруі мүмкін.

4. Алдыңғы fetch сұрауын AbortController арқылы тоқтату немесе әр сұрауға реттік нөмір беріп, тек соңғы жауапты көрсету. Екеуін бірге қолдануға да болады.

5. Жоқ. Жазба Id және ағымдағы UserId шартымен табылуы керек. Рөлге не жазба иесіне негізделген нақты рұқсат әр сұрауда тексеріледі.

Келесі қадам: жобаңызды басқа адамға түсіндіріп көріңіз. Қандай кіріс қабылданады, қандай ереже қорғалады және нәтижені қалай тексересіз?